LAB 2 ADDENDUM · CORE TASKS

Zero Trust: Micro-Segmentation & Admission Control

Egress default-deny and Pod Security Standards — closing the direction an attacker actually travels

Status of This Addendum

Caution: These tasks were previously listed as optional *Expansion Ideas* in Lab 2. They are now **core and assessed**. Complete Tasks Z1 and Z2 after Lab 2 Session B, using the same cluster, and fold the evidence into your Lab 2 report.

Note: This is the first of two Zero Trust addenda. The second — **Lab 4.1: Service Identity with Mutual TLS** — covers Task Z3 and attaches to Lab 4 in Weeks 7–8. Together they cover the three properties described below.

The Idea You Are Testing

Lab 2 built perimeters: separate namespaces, resource quotas, and a default-deny **ingress** policy between tenants. Every one of those controls answers the question *where is this traffic coming from?* Zero Trust rejects that question. Location is not a credential. A packet from inside your cluster deserves no more trust than a packet from the internet, because an attacker who compromises one pod is now inside the perimeter — and everything you built stops helping.

Three properties distinguish a Zero Trust design from the defence-in-depth you already have:

- **Deny by default in both directions.** You built default-deny ingress in Lab 2. Egress is the direction that matters to an attacker who is already inside and wants to exfiltrate data or call home. **(Task Z1, below.)**
- **Verify what the workload is, not just where it sits.** A privileged container shares the host kernel with every other tenant, so namespace boundaries do not contain it. **(Task Z2, below.)**
- **Bind authorisation to a cryptographic identity, not an address.** *(Task Z3 — see the Lab 4 addendum.)*

Course & Assessment Mapping

Item	Mapping
Extends	Lab 2 — Secure Isolation & Multi-Tenancy (Weeks 3–4)
Course Learning Outcome	CLO2 — Construct secure cloud operations (VBE3)
Lecture topics	Week 3 (Secure Isolation of Physical & Logical Infrastructure) · Week 9 (Security Design Patterns II)
CSA CCSK v5 domains	Domain 7 (Infrastructure & Networking) · Domain 12 (Related Technologies — Zero Trust)
Assessment	Folded into the **Lab 2 report** under the existing evidence quality and conceptual understanding criteria

Prerequisite	Your Lab 2 kind cluster with the `tenant-a` and `tenant-b` namespaces still running

Technical Prerequisites

- The kind cluster from Lab 2, with a CNI that enforces NetworkPolicy (Calico, as configured in Lab 2 Session A).
- `kubectl` on your PATH.
- Both tenant namespaces populated with the `api` service from Lab 2, Task 1.

> **Security tip:** If you tore your Lab 2 cluster down, rebuild it from Lab 2 Session A before starting. These tasks depend on the cross-tenant service you created there, and the before/after contrast is the whole point.

Task Z1 — Egress Default-Deny

In Lab 2 you blocked traffic *into* each tenant. An attacker who lands in tenant-a does not care: they want to reach out — to a database, to another tenant, to a command-and-control host. Close that direction.

First, confirm the default. Egress is unrestricted even with your Lab 2 ingress policy in place.

```
# Baseline: a pod in tenant-a can reach anything it likes
kubectl -n tenant-a run probe --rm -it --restart=Never --image=busybox:1.36 -- \
  sh -c "wget -qO- --timeout=3 http://api.tenant-b.svc.cluster.local || echo BLOCKED"
```

Now deny all egress, then permit only what the workload genuinely needs — DNS, and one named service.

```
cat > egress-policy.yaml <<'YML'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
  namespace: tenant-a
spec:
  podSelector: {}
  policyTypes:
    - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-and-api-only
  namespace: tenant-a
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
```

```
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    - to:
        - podSelector:
            matchLabels:
              app: api
      ports:
        - protocol: TCP
          port: 80
YML

kubectl apply -f egress-policy.yaml
kubectl -n tenant-a get networkpolicy
```

```
# Re-test: cross-tenant egress should now fail
kubectl -n tenant-a run probe --rm -it --restart=Never --image=busybox:1.36 -- \
  sh -c "wget -qO- --timeout=3 http://api.tenant-b.svc.cluster.local || echo BLOCKED"

# But the permitted in-namespace service still works
kubectl -n tenant-a run probe --rm -it --restart=Never --image=busybox:1.36 -- \
  sh -c "wget -qO- --timeout=3 http://api.tenant-a.svc.cluster.local || echo BLOCKED"
```

> **Note:** The first egress rule exists only to allow DNS. Omit it and every hostname lookup in the namespace fails, which students almost always diagnose as 'the network policy is broken'. It is not — a default-deny egress policy blocks DNS like anything else, and this is the single most common operational surprise when teams adopt egress control. Record both probe results.

For your report: try deliberately removing the DNS rule and re-running the second probe. Capture the failure, then restore the rule. The difference between *no route* and *no name resolution* is one you will diagnose for the rest of your career.

Task Z2 — Admission Control: Refuse the Workload Outright

Network policy governs what a pod may talk to. It says nothing about what the pod *is*. A privileged container shares the host kernel with every other tenant on the node, so a container escape defeats the namespace isolation you built in Lab 2 entirely. Pod Security Standards refuse such a workload at admission — before it ever runs.

```
kubectl label namespace tenant-a \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=latest \
  pod-security.kubernetes.io/warn=restricted \
  --overwrite

kubectl get namespace tenant-a --show-labels
```

```
cat > privileged-pod.yaml <<'YML'
apiVersion: v1
kind: Pod
metadata:
  name: privileged-probe
  namespace: tenant-a
spec:
  containers:
    - name: probe
      image: busybox:1.36
      command: ["sleep", "3600"]
      securityContext:
        privileged: true
YML

# Expect: rejected by the admission controller, not merely restricted at runtime
kubectl apply -f privileged-pod.yaml
```

> **Security tip:** Read the rejection message carefully — it names the specific `restricted` requirements the pod violated (`privileged`, `allowPrivilegeEscalation`, `runAsNonRoot`, `seccompProfile`, dropped capabilities). This is a **preventative** control: the bad workload never exists, so there is nothing to detect, contain or remediate. Compare that with the container hardening you will do by hand in Lab 4, Task 6 — same outcome, but enforced by the platform rather than by the diligence of whoever wrote the manifest.

Now prove the policy is not simply blocking everything. Deploy a compliant pod and confirm it is admitted.

```
cat > compliant-pod.yaml <<'YML'
apiVersion: v1
kind: Pod
metadata:
  name: compliant-probe
  namespace: tenant-a
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: probe
      image: busybox:1.36
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop: ["ALL"]
YML

kubectl apply -f compliant-pod.yaml
kubectl -n tenant-a get pod compliant-probe
```

In your Lab 2 report, state which of the five `restricted` violations you consider most severe in a multi-tenant cluster, and explain concretely what an attacker achieves by exploiting it.

Deliverables — Add to Your Lab 2 Report

1. Evidence (label each clearly)

- Both cross-tenant egress probes — reachable before the policy, `BLOCKED` after (Z1).
- The in-namespace probe still succeeding, proving the policy permits what it should (Z1).
- The DNS-rule-removed failure, showing name resolution breaking independently of routing (Z1).
- `kubectl -n tenant-a get networkpolicy` listing both policies (Z1).
- The namespace labels showing `enforce=restricted` (Z2).
- The admission controller's rejection message, with the violated requirements named (Z2).
- The compliant pod being admitted and running (Z2).

2. Short-Answer Questions

1. Lab 2 gave you default-deny ingress. Explain why default-deny **egress** is the control an attacker actually cares about, and name two specific things they can no longer do once it is in place.
2. Your first egress policy broke every hostname lookup in the namespace. Explain why at the protocol level, and state what this implies about testing a deny-by-default control before shipping it to production.
3. Pod Security Standards rejected the privileged pod at admission. Contrast that with detecting a privileged pod after it has started: what does the preventative control give you that the detective one cannot?
4. Namespaces gave you isolation in Lab 2. Explain why a privileged container defeats that isolation, and identify what the two workloads are actually sharing.
5. Zero Trust is often summarised as 'never trust, always verify'. Using one example each from Z1 and Z2, state what is being verified and what assumption is being refused.

3. Verification Command

```
echo "=== Lab 2 Addendum verification ==="
kubectl -n tenant-a get networkpolicy -o custom-
columns=NAME:.metadata.name,TYPES:.spec.policyTypes
kubectl get namespace tenant-a -o jsonpath='{.metadata.labels}' | tr ',' '\n' | grep
pod-security
kubectl -n tenant-a get pods
```

Security Best-Practices Checklist

- ☐ Egress is denied by default, not only ingress.
- ☐ Permitted egress is an explicit allow-list, including a deliberate DNS rule.
- ☐ Cross-tenant traffic was tested and is blocked in both directions.
- ☐ The namespace enforces the `restricted` Pod Security Standard at admission.
- ☐ A privileged workload is rejected before it runs, not detected after.
- ☐ A compliant workload still deploys — the control is scoped, not a blanket block.

Cleanup

```
kubectl delete -f egress-policy.yaml --ignore-not-found
kubectl delete pod compliant-probe -n tenant-a --ignore-not-found
kubectl label namespace tenant-a \
  pod-security.kubernetes.io/enforce- \
  pod-security.kubernetes.io/enforce-version- \
  pod-security.kubernetes.io/warn- 2>/dev/null
rm -f egress-policy.yaml privileged-pod.yaml compliant-pod.yaml
```

References

- Course lectures — Week 3 (Secure Isolation of Physical & Logical Infrastructure); Week 9 (Security Design Patterns II).
- Kubernetes Network Policies, including egress — kubernetes.io/docs/concepts/services-networking/network-policies
- Kubernetes Pod Security Standards — kubernetes.io/docs/concepts/security/pod-security-standards
- NIST SP 800-207, Zero Trust Architecture — csrc.nist.gov/publications/detail/sp/800-207/final
- CSA Security Guidance v5 — Domain 7 (Infrastructure & Networking) and Domain 12 (Zero Trust).
- Companion document — **Lab 4 Addendum: Zero Trust Service Identity with Mutual TLS** (Task Z3).